Dynamic Food-Readiness-Aware Priority Routing (DFPR)

Data Structures Project Report

Submitted by

Simar Bhatia (26MCS0020)

Yash Mohadikar (26MCS0037)

Amandeep Singh Bhatti (26MCS0046)

M.Tech (CSE), Semester I

1. Introduction

In food delivery, a dispatcher must decide which free rider picks up which waiting order. Common rules either send the rider to the nearest restaurant or serve the order with the earliest deadline. Both ignore when the food will actually be ready, so riders often stand at restaurants waiting for food, while elsewhere cooked food goes cold waiting for a rider. DFPR (Dynamic Food-Readiness-Aware Priority Routing) ranks every waiting order with a priority score that combines urgency, food readiness, waiting time, distance, traffic and the rider's predicted waiting time, and updates that score whenever something changes in the city.

2. The DFPR Priority Score

For each waiting order i at time t, using the nearest free rider (by traffic-aware travel time):

$$\mathbf{P(i,t) = wU*U + wF*F + wW*W - wD*D - wC*C - wR*R}$$

- U (urgency): grows as the predicted delivery time approaches the deadline.
- F (food readiness): highest when the rider arrives exactly when the food is ready.
- W (waiting): how long the order has been waiting since it was placed.
- D (distance): road distance rider -> restaurant -> customer.
- C (traffic): extra travel time caused by traffic on both legs.
- R (rider wait): predicted time the rider would stand at the restaurant.

Every factor is normalised to the range [0, 1]. The six weights were tuned by random search on 10 development days (best: wU=0.50, wF=0.05, wW=1.80, wD=0.48, wC=2.30, wR=2.16) and evaluated on 30 different held-out days.

3. Data Structures Used

All core data structures were implemented from scratch in Python (no heapq or networkx in the core code):

Data structure	**Used for**	**Cost**
Indexed binary max-heap (+ position hash map)	Priority queue of waiting orders; scores change in place	push / pop / update_key / remove: O(log n)
Binary min-heap	Event queue of the discrete-event simulation	push / pop: O(log n)
Weighted graph (adjacency list)	Road network with per-road traffic multipliers	neighbour scan: O(degree)
Dijkstra / A* search	Shortest (fastest) routes for riders	O((V + E) log V)
Route cache	Reuses shortest-path trees; drops only those affected by traffic	lookup O(1) on a hit
Hash maps (dict)	Order, rider and restaurant state	lookup / update O(1)

4. How the System Works

- City: a 25 x 25 street grid with diagonal main roads, 40 restaurants and clustered customers; traffic varies by time of day with random jams.
- Orders arrive randomly (with lunch and dinner peaks). Each has an estimated cooking time; the real cooking time is noisy and only known when the food is ready.
- Event-driven simulator: order placed, food ready, rider arrived, pickup, delivered, rider free, traffic change. After each event the dispatcher decides which free rider takes which order.
- DFPR keeps all waiting orders in the indexed heap and re-scores only the orders an event affects (food ready, traffic change, rider becomes free).
- Baselines compared on exactly the same simulated days: Nearest Order First, Shortest-Path Dispatch, Static Priority (earliest deadline first), plus FIFO and a traffic-aware nearest-pair control.
- An interactive Streamlit demo application replays recorded runs; the screenshots below are taken from it.

5. Demo Application: Screenshots and Explanation

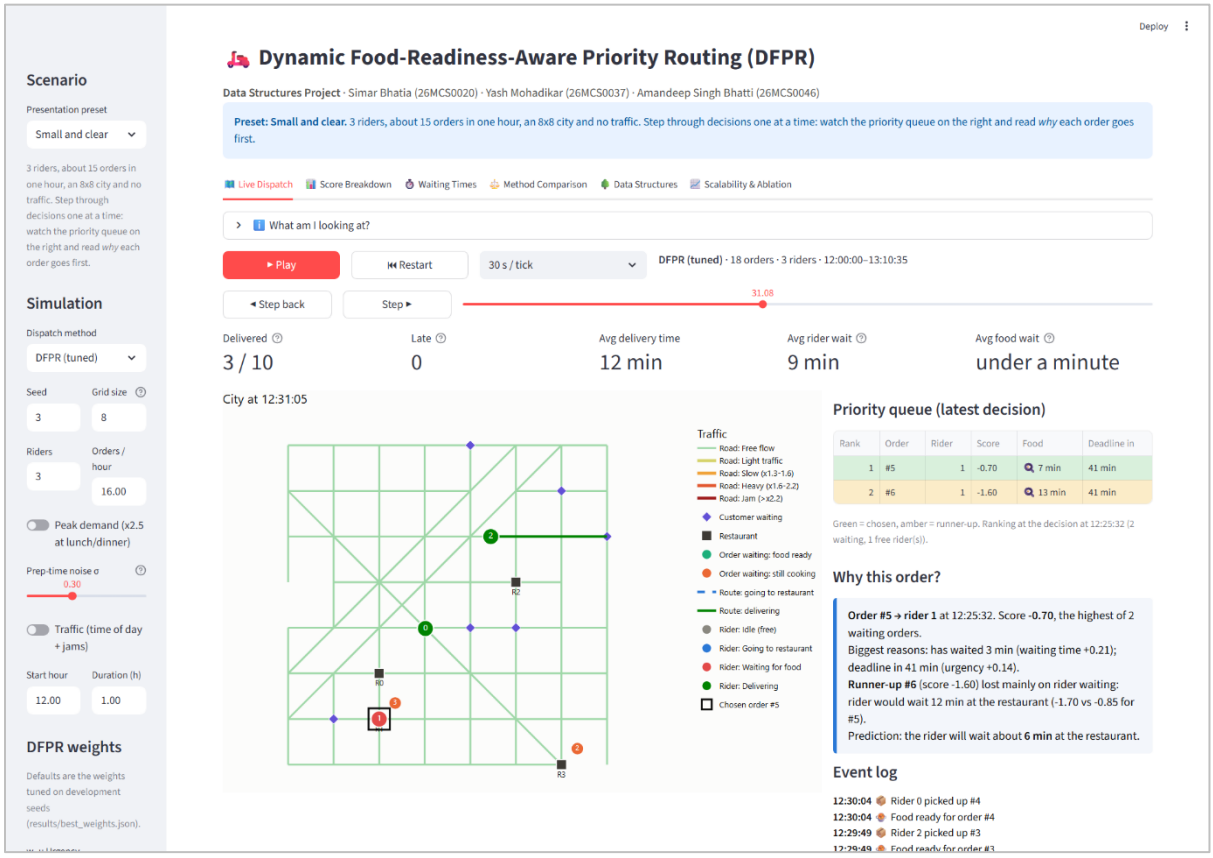


Figure 1: Live Dispatch tab, 'Small and clear' preset (3 riders, 8 x 8 city).

The main screen. The header shows the project title and team. Left: the city map, where black squares are restaurants, purple diamonds are waiting customers and numbered circles are riders coloured by status (grey free, blue going to a restaurant, red waiting for food, green delivering). The small orange/green circle at a restaurant shows an order still cooking or ready. Right: the priority queue at the latest decision (green row = chosen order), the plain-English 'Why this order?' explanation and the event log. KPI cards on top show orders delivered, late orders, average delivery time and average waiting times so far.

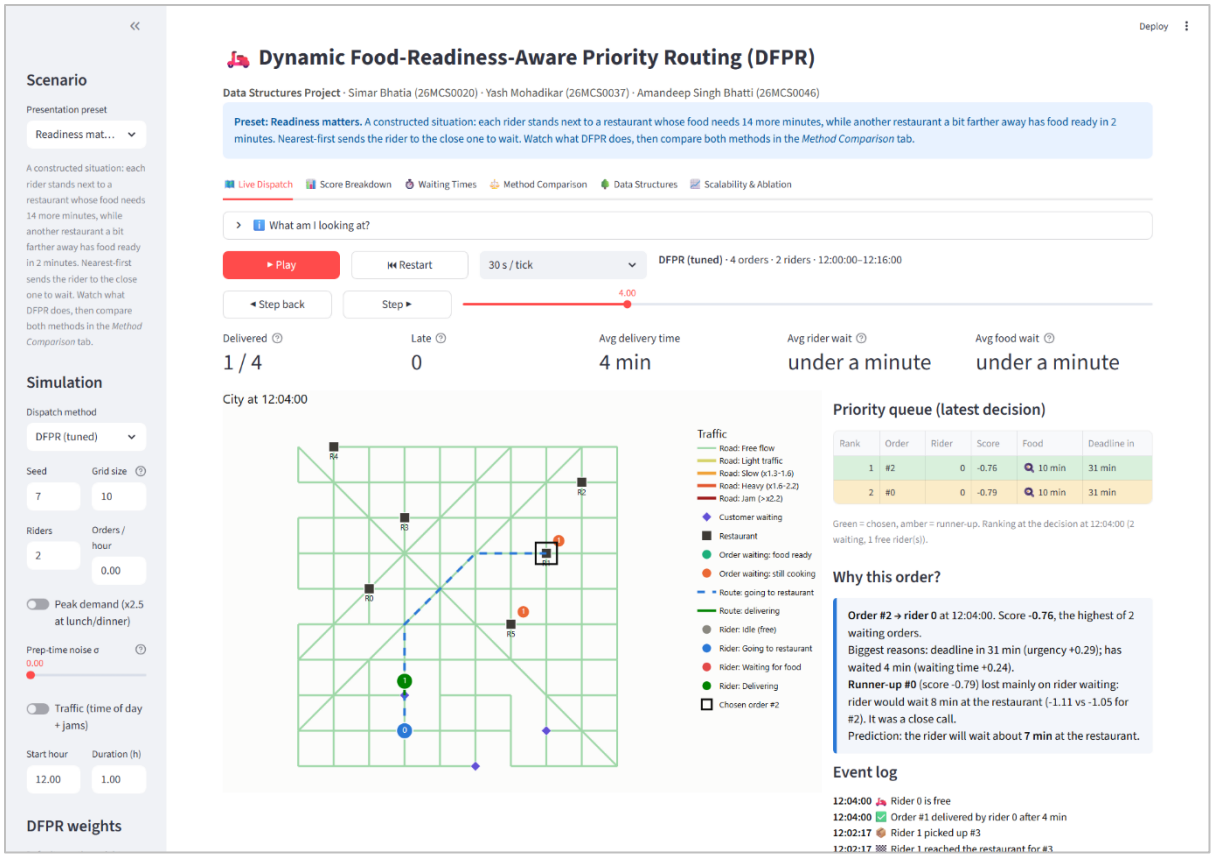


Figure 2: 'Readiness matters' preset: the decision DFPR makes.

A constructed situation: each rider stands next to a restaurant whose food needs 14 more minutes, while a restaurant farther away has food ready in 2 minutes. The queue and explanation show DFPR's reasoning: the runner-up order lost mainly on rider waiting, because the rider would have waited 8 minutes at the restaurant. DFPR therefore sends the rider (blue dashed route) to the order that will be ready on arrival.

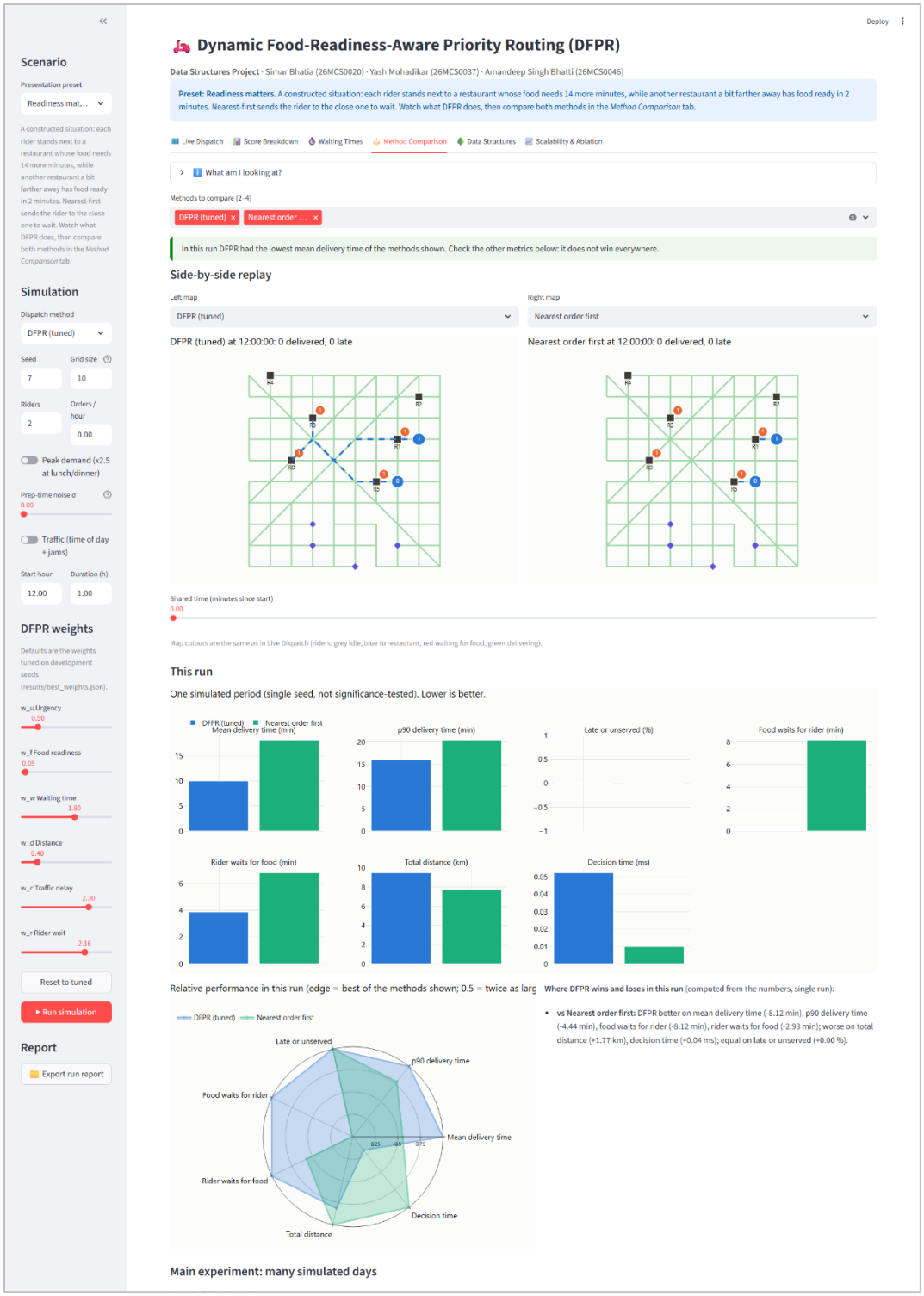


Figure 3: Method Comparison tab: DFPR vs Nearest Order First on the same scenario.

Both methods run on the identical simulated period. The side-by-side maps share one time slider. In this scenario DFPR's mean delivery time is about 10 minutes against about 18 for Nearest Order First, because Nearest-first parks each rider for 14 minutes. The bars, radar chart and the

automatically generated summary also show where DFPR loses: it drives more distance here. The green banner is computed from the numbers, not hard-coded.

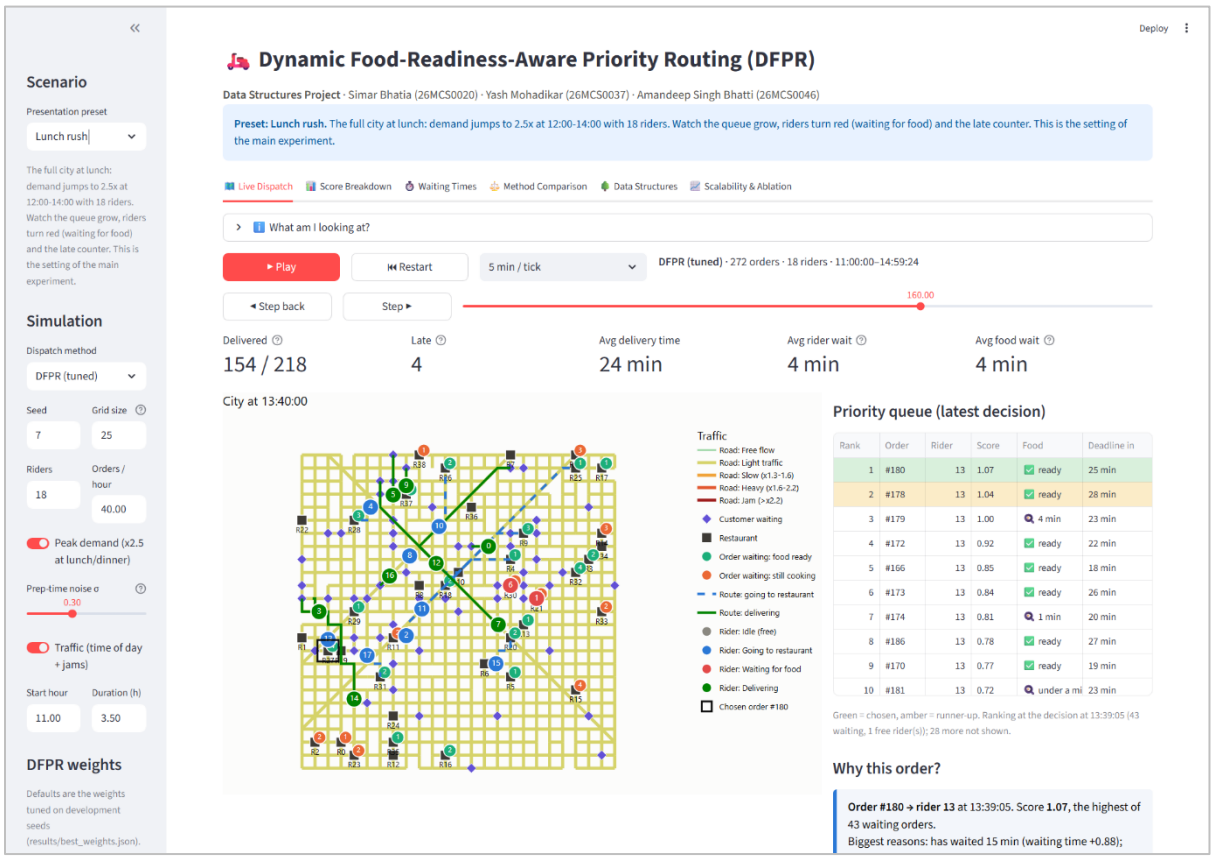


Figure 4: 'Lunch rush' preset at 13:40 (18 riders, demand x2.5 at 12:00-14:00).

The full city during the peak. 43 orders are waiting and only one rider is free, so the ranking matters. Roads are yellow because lunch traffic slows them. The queue shows each order's score, whether its food is ready and the time left to its deadline; DFPR picks order #180, whose food is ready and which has waited 15 minutes.

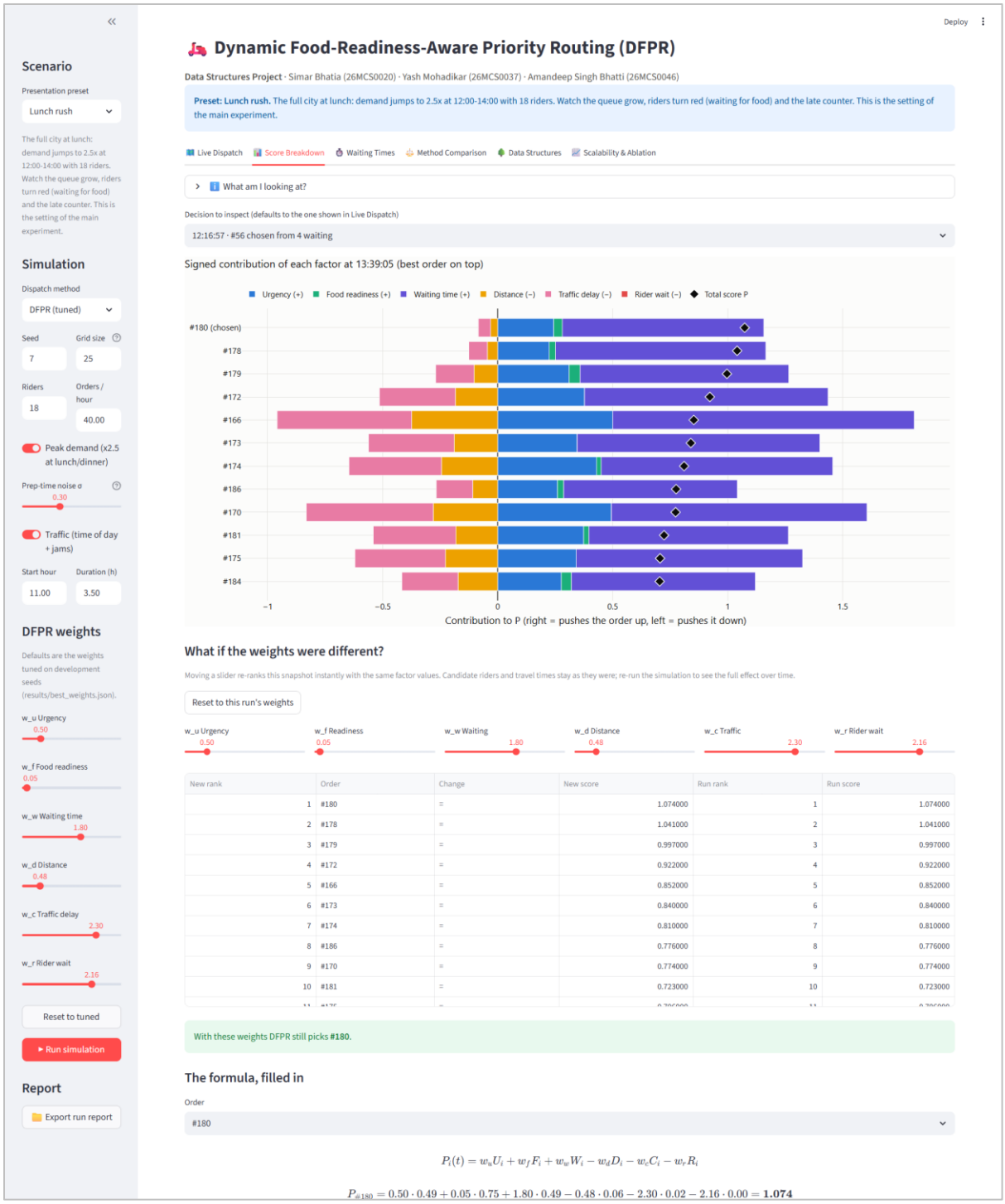


Figure 5: Score Breakdown tab.

Each bar shows how much every factor adds to (right) or subtracts from (left) an order's score; the black diamond is the total. The 'what-if' sliders re-rank the same snapshot instantly with different weights (arrows show rank changes), and the formula at the bottom is filled in with the selected order's real numbers.



Figure 6: Waiting Times tab.

Top: a timeline for each order (grey = waiting for a rider, blue = rider driving to the restaurant, red = rider waiting for food, orange = food waiting for the rider, green = delivering; black tick = deadline). Below: distributions of rider waiting, food waiting and delivery time, and two panels on one time axis showing how the arrival rate peaks at lunch and how average waits respond.

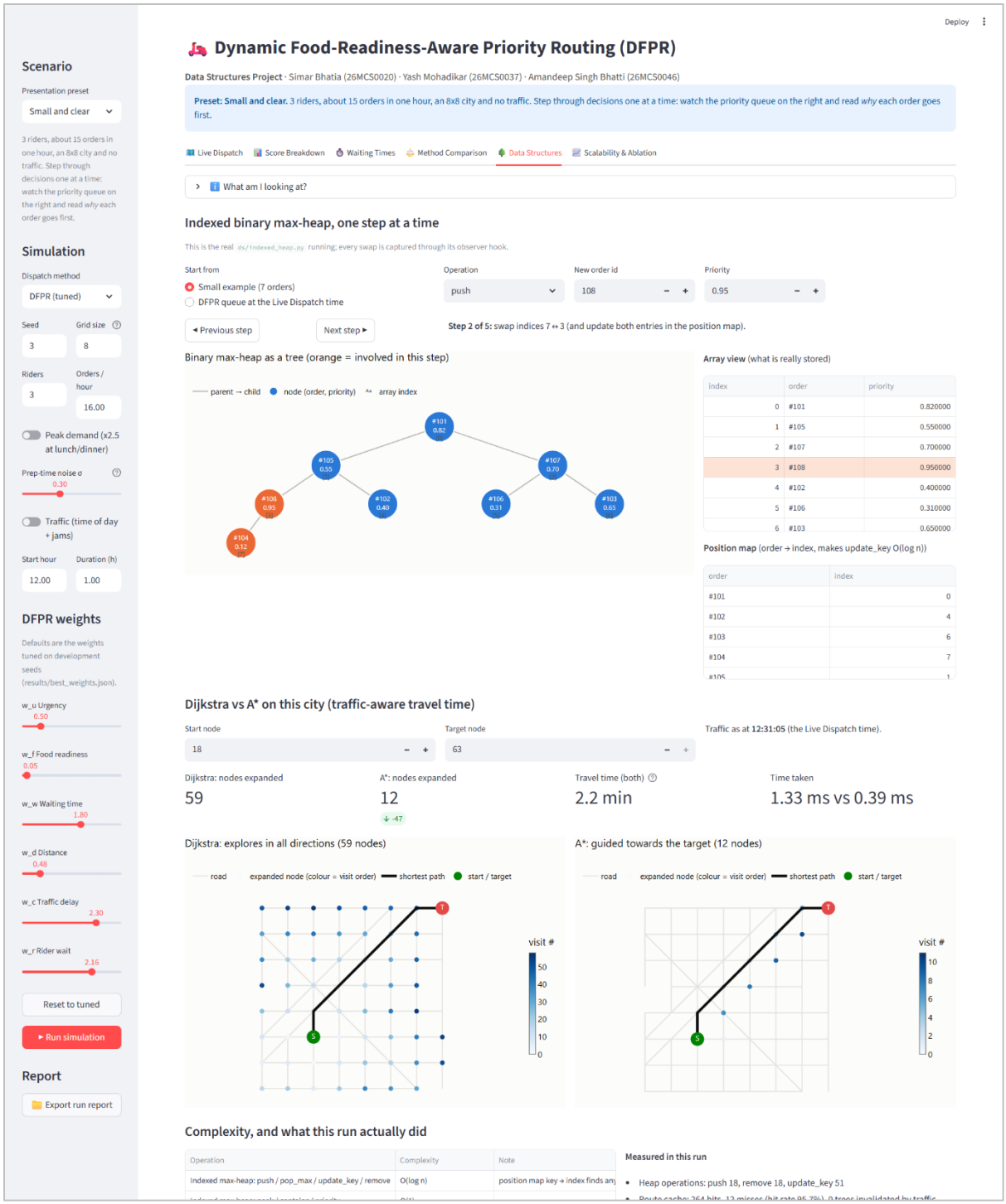


Figure 7: Data Structures tab.

Top: a step-by-step run of the real indexed max-heap (here a push of order #108); the tree, the array and the position hash map update together, and orange nodes are the ones being swapped. Middle: Dijkstra vs A* on the same route; both find the same 2.2-minute path, but A* expands 12 nodes instead of 59. Bottom: Big-O complexity table and the counts measured in the run (heap operations, route-cache hit rate).

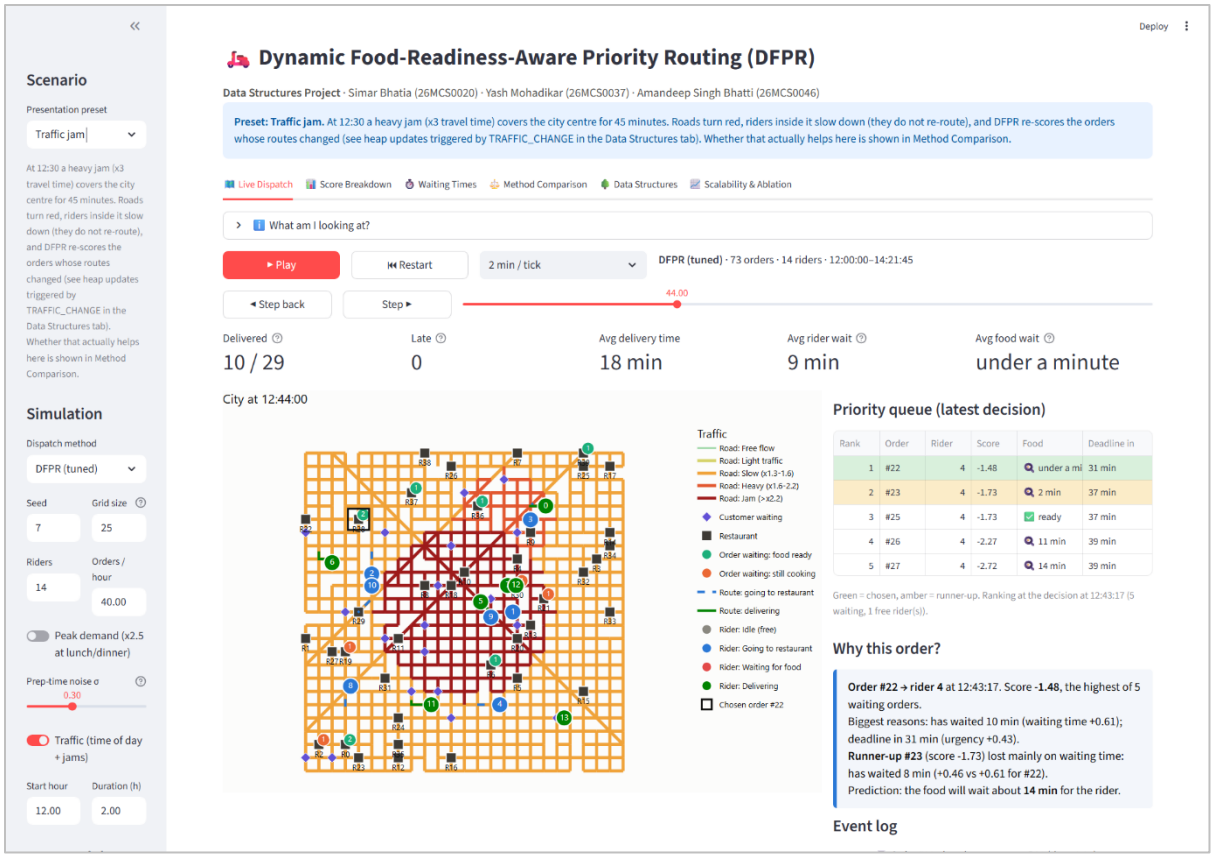


Figure 8: 'Traffic jam' preset.

A heavy jam (x3 travel time) over the city centre is drawn in red. Riders inside it slow down, and DFPR re-scores the orders whose routes changed. In this scenario DFPR is not better than Nearest Order First on mean delivery time (23.8 vs 23.2 min); the app reports this honestly in the comparison tab.

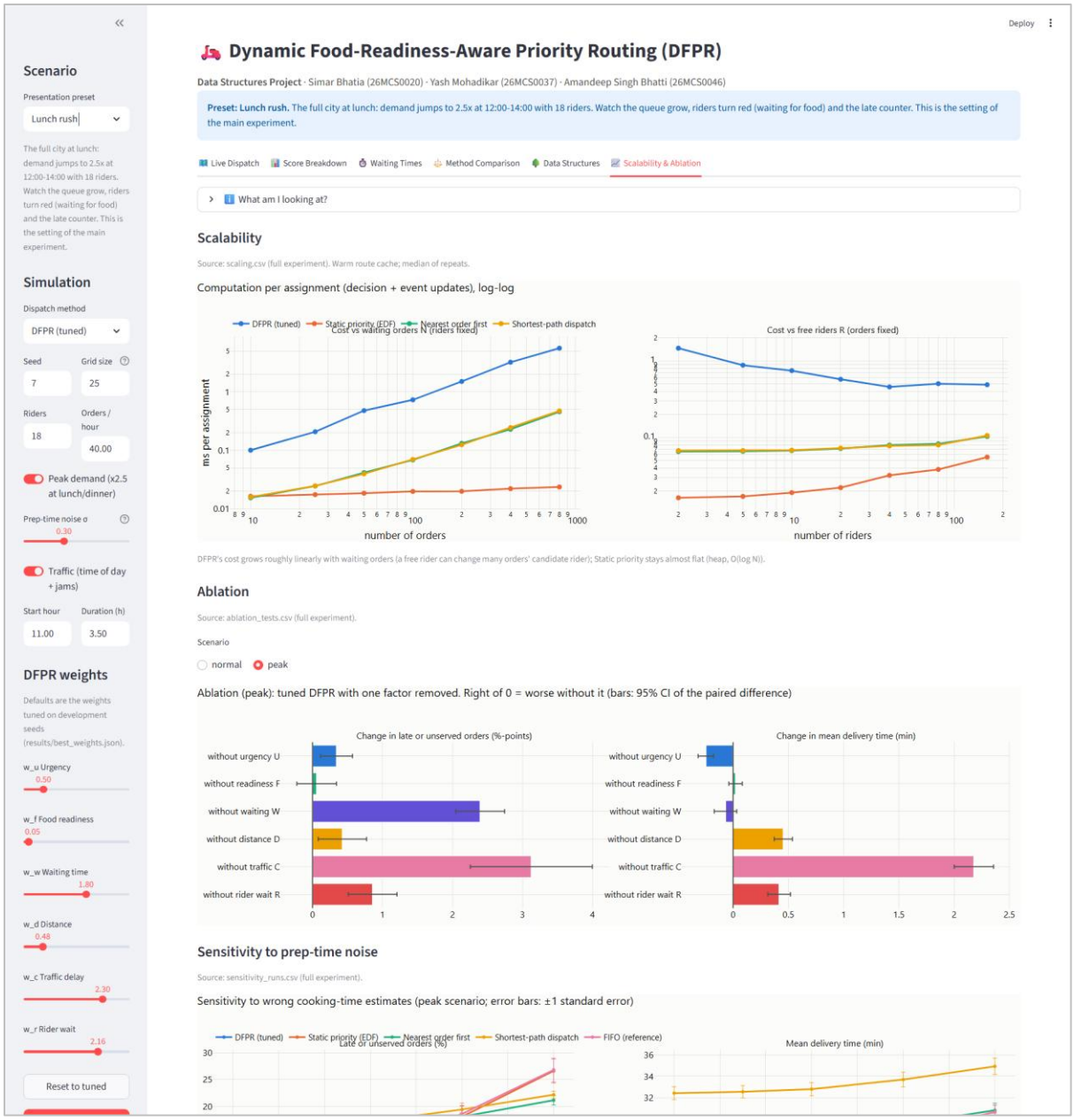


Figure 9: Scalability & Ablation tab (results of the batch experiments).

Scaling: computation per assignment versus the number of waiting orders and free riders (DFPR grows roughly linearly with waiting orders; Static Priority stays almost flat). Ablation: tuned DFPR with one factor removed; waiting time (W) and traffic delay (C) matter most, while readiness alignment (F) has no significant effect. Sensitivity: late deliveries as cooking-time estimates become noisier.

6. Main Results (30 held-out days, peak demand)

Method	Late or unserved (%)	Mean delivery (min)	p90 delivery (min)
DFPR (tuned)	**2.97**	**23.2**	**33.5**

FIFO (reference)	12.76	27.1	41.7
Static Priority (EDF)	13.27	27.4	42.5
Nearest pair (traffic-aware)	15.25	28.6	56.8
Nearest Order First	15.52	28.7	57.1
Shortest-Path Dispatch	17.03	32.8	77.0

At peak demand DFPR was better than every baseline on late deliveries, mean and p90 delivery time on all 30 held-out days (paired Wilcoxon test, Holm-corrected $p < 0.001$). Under normal demand all methods are practically tied (about 0.6% late), because cooking time, not dispatching, is the bottleneck. DFPR loses on some measures: riders wait somewhat longer at restaurants, it drives more distance than the nearest-first rules, and it needs more computation (about 10-14 ms per order for its dynamic updates).

7. Conclusion and Limitations

DFPR shows that a dynamically updated priority score, maintained in an indexed heap and fed by shortest-path routing, can sharply reduce late deliveries in busy periods. The data structures make each update cheap ($O(\log n)$ per order) and the route cache avoids repeated Dijkstra runs. Limitations: the city, demand and cooking times are simulated; riders carry one order at a time; only one load level was studied; and the readiness-alignment factor F did not help in the experiments; the benefit came mainly from waiting time, traffic delay and rider-wait terms.